Code Explanation

Unit Tests for Workflow Runner

The provided code is a set of unit tests for a workflow runner written in Python. It utilizes the Pytest framework for testing and includes tests for the `get_config`, `setup_logging`, and `main` functions.

Overview

The code is designed to test the functionality of a workflow runner. It includes tests for configuration retrieval, logging setup, and the main execution function. The tests cover various scenarios, including successful execution and failure handling.

Step 1: Importing Necessary Modules and Setting Up the Test Environment
The code begins by importing the necessary modules, including Pytest, OS, and Sys. It also imports the `MagicMock` class from the `unittest.mock` module for mocking objects. The `SparkSession` class from `pyspark.sql` is also imported.

```
import pytest
import os
import sys
from unittest.mock import patch, MagicMock
from pyspark.sql import SparkSession
```

Step 2: Adding the Src Directory to the Path
The code adds the `src` directory to the system path to allow importing modules from it.

```
sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file_
 ), '..')))
```

Step 3: Defining a Fixture for a Mock Spark Session
A Pytest fixture named `mock_spark_session` is defined to create a mock `SparkSession` object. This fixture is used to mock the `SparkSession` builder and return a mock session.

```
@pytest.fixture
def mock_spark_session():
    """Create a mock SparkSession."""
    mock_session = MagicMock()
    mock_builder = MagicMock()
    mock_builder.appName.return_value = mock_builder
    mock_builder.enableHiveSupport.return_value = mock_builder
    mock_builder.config.return_value = mock_builder
    mock_builder.getOrCreate.return_value = mock_session

    with patch('pyspark.sql.SparkSession.builder', mock_builder):
        yield mock_session
```

Step 4: Testing the Get Config Function

The `test_get_config` function tests the `get_config` function by checking its return values with default and custom configurations.

```
def test_get_config():
    """Test the get_config function."""
    config = get_config()
    assert config["target_dir"] == "/dbfs/mnt/target/files"
    assert config["batch_size"] == 10000

    with patch.dict(os.environ, {"TARGET_FILE_DIR": "/custom/path",
"BATCH_SIZE": "5000"}):
        config = get_config()
        assert config["target_dir"] == "/custom/path"
        assert config["batch_size"] == 5000
```

Step 5: Testing the Setup Logging Function
The `test_setup_logging` function tests the `setup_logging` function by verifying that it creates a log directory, configures logging, and returns a logger.

```
def test_setup_logging():
    """Test the setup_logging function."""
    with patch('logging.FileHandler') as mock_file_handler,
         patch('logging.StreamHandler') as mock_stream_handler,
         patch('logging.basicConfig') as mock_basic_config,
         patch('os.makedirs') as mock_makedirs:

        logger = setup_logging()

        mock_makedirs.assert_called_once()
        mock_basic_config.assert_called_once()
        assert logger is not None
```

Step 6: Testing the Main Function with Success
The `test_main_success` function tests the `main` function with a successful execution by verifying that logging is set up, the workflow is run, and success is logged.

```
@patch('src.workflow_runner.run_workflow')
def test_main_success(mock_run_workflow, mock_spark_session):
    """Test successful execution of the main function."""
    with patch('src.workflow_runner.setup_logging') as mock_setup_lo
gging:
        mock_logger = MagicMock()
        mock_setup_logging.return_value = mock_logger

        result = main()

        mock_setup_logging.assert_called_once()
        mock_run_workflow.assert_called_once()
        mock_logger.info.assert_any_call("Workflow completed success
fully")
        assert result == 0
```

Step 7: Testing the Main Function with Failure
The `test_main_failure` function tests the `main` function with a failure by verifying that the error is logged and the correct return code is returned.

```
@patch('src.workflow_runner.run_workflow')
def test_main_failure(mock_run_workflow, mock_spark_session):
    """Test handling of failures in the main function."""
    with patch('src.workflow_runner.setup_logging') as mock_setup_lo
gging:
        mock_logger = MagicMock()
        mock_setup_logging.return_value = mock_logger

        mock_run_workflow.side_effect = Exception("Test error")

        result = main()

        mock_logger.error.assert_called_once()
        assert result == 1
```